MIPHI · POWERED BY PHISON

CrewAI with aiDAPTIV+

Teams of AI agents that run on your own hardware and remember what they have already read.

User Manual and Setup Guide

01

Why it matters

What CrewAI and aiDAPTIV+ do, and what your business gets out of them.

02

Setting it up

Step by step instructions for creating and running crews with aiDAPTIV+ on your own hardware.

03

Checking it works

How to confirm on your own system that aiDAPTIV+ is saving time.

Version 1.0 · October 2026 · Prepared by MiPhi

Contents

Why CrewAI with aiDAPTIV+ 3

A typical example 3
What CrewAI is 3
What aiDAPTIV+ adds 4
What happens when a crew runs 4
Benefits 4
Test results 5
Where it helps most 6
How the setup is organised 7

Part 1: Before You Begin 8

What you need 8
Start the model server 8
Cloning our CrewAI repository 9

Part 2: The aiDAPTIV Cache Configuration 10

Why crew memory is turned off 11

Part 3: Installing CrewAI and Creating a Crew 12

Check the Python version 12
Install uv 12
Install CrewAI 12
Set up the PATH and check the version 13
Create a crew 13
Look inside the crew folder 16
Connect the agent to aiDAPTIV+ 18

Part 4: Running a Crew 19

Run your new crew 19
Run the example crew with a test document 21

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working 24

Check the cache folder on disk 24
Confirm a cache hit 25

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit 25

Run 2: Cache Hit (KV Reuse) 25
Benchmark your own system 26

Part 7: Troubleshooting 27

Why CrewAI with aiDAPTIV+

Most teams we speak to would like AI to take on real work with their own documents, but two things hold them back. The first is privacy. The well-known AI services run in someone else's cloud, so every document has to be sent there first. The second is speed. Give an AI a long document and you can wait several minutes before the first word of the answer shows up, and a crew of AI agents goes back to the model again and again.

CrewAI with aiDAPTIV+ was put together to deal with both. This chapter explains how, in plain terms. If you would rather go straight to the installation, skip ahead to the Setup Guide.

A typical example

Say your team uses a crew to explain a long technical report in simple terms. In our tests, a document of about 12,400 tokens kept the crew waiting 4 minutes 38 seconds before the model wrote its first word, because it had to read every page first.

Then someone runs the crew again on the same report. Without aiDAPTIV+, the model reads all of it again and the wait starts over.

With aiDAPTIV+, the model keeps what it learned the first time. In the same test the repeat run started answering in **about 1.2 seconds**, even though the server had been restarted in between. And the report never left your own machine.

What CrewAI is

CrewAI is an open-source framework for building teams of AI agents. Each agent is given a role, a goal and a short backstory, and the crew works through a list of tasks. You describe your agents and tasks in simple settings files and start the crew with one command.

Agents with a clear job

Each agent has a role, a goal and a backstory, so it behaves like a specialist rather than a general chatbot.

Tasks in order

A crew works through its tasks one after another, and each agent can build on what the previous one produced.

Plain settings files

Agents and tasks live in readable .jsonc files, so changing a role or a prompt is a text edit, not a code change.

Runs from a terminal

One command, crewai run, starts the crew and shows its progress on a live screen, with the final result at the end.

What aiDAPTIV+ adds

When an AI model reads a document, it builds up a set of working notes about the text. Engineers call these notes the **KV cache**. They are normally kept in the graphics card's memory, which is small and expensive, so they get thrown away when space runs out or the server restarts. The next request then has to start from the beginning.

aiDAPTIV+, made by Phison, keeps those notes on a fast SSD instead. There is far more room there, and nothing gets thrown away. When a new request involves material the model has already read, it loads its notes and carries on.

> **A SIMPLE WAY TO THINK ABOUT IT**
>
> Picture a librarian who answers questions about a book. Without aiDAPTIV+, they read the whole book again every time someone asks something. With aiDAPTIV+, they keep their notes on the shelf and can answer the next person almost straight away, even if the library closed overnight in between.

What happens when a crew runs

Your team never needs to think about any of this, but if you are curious, this is the sequence:

1
You start the crew
Someone runs crewai run in a terminal, with the document and question set in the crew's inputs.

→

2
The agent asks the model
Each agent sends its instructions and the document to the AI model on your own hardware.

→

3
aiDAPTIV+ checks its notes
Before reading anything again, aiDAPTIV+ looks on the SSD for notes it already has.

→

4
The result comes back
The agent's answer appears in the terminal, and any new notes are saved for next time.

Benefits

Benefit	In practice
Your data stays in-house	The model, your documents and everything the crew produces stay on your own hardware. Nothing is sent to an outside AI provider.
Faster repeat runs	Running a crew again on the same documents reuses earlier work, so the model starts answering in seconds rather than minutes.

Benefit	In practice
Quicker even the first time	In our tests, the very first read of a new document was also about twice as fast with aiDAPTIV+.
Survives restarts	The notes sit on the SSD rather than in temporary memory, so the model still remembers what it has read after the server restarts.
Better use of existing hardware	Long documents and long instructions no longer have to fit in the graphics card's memory, because the SSD takes the overflow.
Predictable cost	There are no per-call or per-token cloud charges, which matters for agents because they call the model many times.
Simple setup	CrewAI installs with a few commands, and a crew connects to aiDAPTIV+ by adding one block to its agent file.
A ready-made example	Our repository includes an example crew and three test documents, so you can see the difference before you build your own.

Test results

We ran the same three documents through a CrewAI crew on the same PC, once without aiDAPTIV+ and twice with it: a first read, and then a second run after restarting the server. Part 5 of this guide shows you how to check the same things on your own system.

1.2 s

Time to the first word on a 12,400-token document after a server restart. Without aiDAPTIV+ the same prompt took 4 minutes 38 seconds.

164 to 229×

How much sooner the answer started across our three tests when aiDAPTIV+ reused its saved notes.

About 2×

How much faster the very first read of a new document was with aiDAPTIV+ in the same tests.

Time to the first word of the answer

Prompt size	Without aiDAPTIV+	aiDAPTIV+, first read	aiDAPTIV+, after restart
About 8,000 tokens 7,844 tokens	2 min 42 s	1 min 15 s	**1.0 s**
About 10,000 tokens 9,682 tokens	3 min 25 s	1 min 36 s	**0.9 s**
About 13,000 tokens 12,445 tokens	4 min 38 s	2 min 12 s	**1.2 s**

Measured by MiPhi in October 2026 using the gemma-4-26B-A4B model (4-bit) on an Intel Core Ultra 7 PC with integrated graphics and 32 GB of memory. The aiDAPTIV+ runs used the aiDAPTIV+ middleware with an NVMe SSD cache, and the server was restarted between the first and second run. Your results will depend on your hardware, the model and the size of your documents.

WHAT THIS MEANS DAY TO DAY

Without aiDAPTIV+, the largest test kept the crew waiting more than four and a half minutes before the model wrote anything. With aiDAPTIV+, the repeat run started answering in just over a second. A crew that goes back to the same material many times saves that wait on every run.

Where it helps most

Research and analysis teams run the same crew over long reports, papers and market studies, asking a different question each time.

Engineering and R&D teams use agents to review specifications, design documents and test reports that are too sensitive for a public cloud.

Support and service teams build agents that answer from product manuals and troubleshooting guides.

Legal and compliance teams check contracts and policies without the text ever leaving the company.

Developers building automations test and rerun crews many times a day, and every rerun on the same material starts in seconds.

How the setup is organised

The Setup Guide takes you from a fresh machine to your first result. It is split into seven parts:

Part	What you do	Result
Part 1 **Before You Begin**	Check the requirements, start the AI model server and download our CrewAI repository.	Everything CrewAI needs is in place.
Part 2 **The aiDAPTIV Cache Configuration**	Read what the aiDAPTIV+ settings in an agent's llm block do.	Nothing to edit, just background.
Part 3 **Creating a Crew**	Install CrewAI by hand, then create a crew connected to aiDAPTIV+.	A new crew already connected to aiDAPTIV+.
Part 4 **Running a Crew**	Run your new crew, or the example crew with one of our test documents.	Your first result.
Part 5 **Verifying the Cache**	Look at the SSD cache folder and the server log.	Confirmation that aiDAPTIV+ is saving and reusing its notes.
Part 6 **Benchmark**	Check the time to first token when saved work is reused.	Figures you can share with your team.
Part 7 **Troubleshooting**	Look up a message you see during setup.	A fix for the most common problems.

When you are ready, turn the page and start with Part 1.

SETUP GUIDE

Getting it running

CrewAI runs from a Windows terminal and connects to an AI model that is served separately by the aiDAPTIV+ middleware. This guide shows you how to install CrewAI by hand, create a crew that is connected to aiDAPTIV+, and run the example crew from our repository straight away.

Part 1: Before You Begin

What you need

Requirement	Details
Windows PC	Windows 10 or 11. The commands in this guide are for Windows PowerShell.
aiDAPTIV+ middleware	Installed as per its own setup guide, with the aiDAPTIV NVMe cache path configured within the middleware.
AI model server	The aiDAPTIV llama.cpp server, started through the middleware and serving your model (a .gguf file). By default it listens on http://localhost:8080/v1.
Git	Used to download our repository. You can also download it from GitHub as a ZIP file.
Python	Python 3.10 to 3.13 (Python >=3.10 and <3.14). Part 3 shows how to check the version you have.
Internet connection	Needed to download uv and CrewAI, and the crew's dependencies the first time you install a crew.

Start the model server

Start the AI model server through the aiDAPTIV+ middleware and confirm that it reports a listening, OpenAI-compatible address, for example http://localhost:8080/v1. You can check it from a terminal:

```
curl http://localhost:8080/v1/models
```

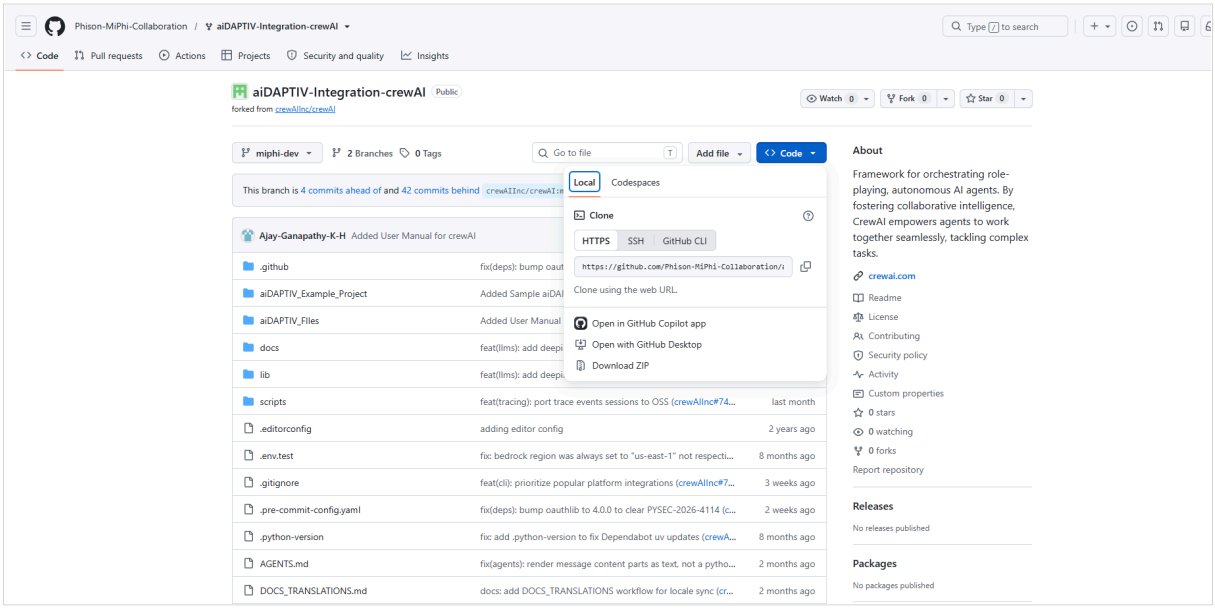
The reply lists the model the server is serving, for example gemma-4-26B-A4B-it-UD-Q4_K_M.gguf. Keep a note of this name, as you can use it in the agent's llm block in Part 3.

> **NOTE**
>
> CrewAI does not load the model itself. It talks to whatever OpenAI-compatible address your aiDAPTIV+ middleware exposes, so get that server running and confirmed first.

Cloning our CrewAI repository

Use our aiDAPTIV-Integration-crewAI repository. The miphi-dev branch includes an example crew that is already set up for aiDAPTIV+, and three test documents of different sizes.

github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-crewAI (miphi-dev branch)



The aiDAPTIV-Integration-crewAI repository on GitHub, with the Code menu open to copy the clone address

1. Open PowerShell, then clone the repository and switch into it:

```
git clone -b miphi-dev https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-crewAI.git
cd aiDAPTIV-Integration-crewAI
```

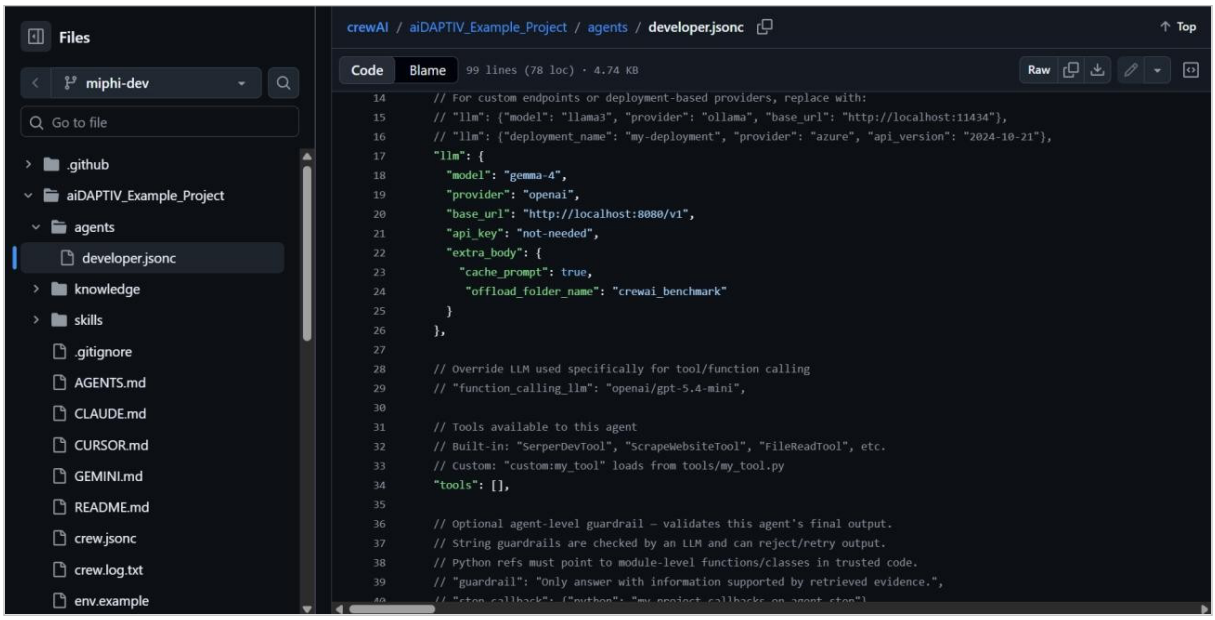
2. The repository has one folder you will use in this guide:

Folder	What it contains
aiDAPTIV_Example_Project	A ready-made crew with one agent, the run.py helper and three test documents: prompt_8k.txt, prompt_10k.txt and prompt_13k.txt (Part 4).

Part 2: The aiDAPTIV Cache Configuration

Every CrewAI agent has a settings file in the agents folder of its crew, for example agents\developer.jsonc. Inside it, the llm block says which model the agent uses and where to find it. For aiDAPTIV+, this block points at the aiDAPTIV llama.cpp server and adds two cache parameters under extra_body.

The example crew already has this block. When you create your own crew in Part 3, you add the same block to its agent file yourself, so this Part explains what each setting does.



The llm block in aiDAPTIV_Example_Project\agents\developer.jsonc

```
"llm": {
  "model": "gemma-4",
  "provider": "openai",
  "base_url": "http://localhost:8080/v1",
  "api_key": "not-needed",
  "extra_body": {
    "cache_prompt": true,
    "offload_folder_name": "crewai_benchmark"
  }
},
```

What each key does:

Key	What it does
model	The model the agent asks for. The safest choice is the exact name your server reports at /v1/models. A server that serves a single model answers whatever name it is given, which is why the short name gemma-4 works in the example crew.
provider	openai. The aiDAPTIV llama.cpp server uses the same API as OpenAI, so CrewAI talks to it through its OpenAI connector. Nothing is sent to OpenAI.
base_url	The address of the aiDAPTIV llama.cpp server, http://localhost:8080/v1 by default.
api_key	A placeholder. The local server does not check it, but CrewAI expects a value.
cache_prompt	Tells the llama-server to save the KV cache generated during prefill instead of discarding it once the reply is sent, and to check for a matching cache before recomputing on the next request.
offload_folder_name	The subfolder, under your aiDAPTIV NVMe cache root, where the KV-cache files are written (crewai_benchmark in the example crew). You will look for this folder again in Part 5.

CrewAI adds everything under extra_body to each request it sends to the model server, so the two cache parameters reach aiDAPTIV+ on every call the agent makes.

Why crew memory is turned off

The crew settings file, crew.jsonc, contains "memory": false. CrewAI's memory feature needs an embedding model, and by default it calls OpenAI's online service for that rather than your aiDAPTIV+ server. Leaving memory off keeps every call on your own hardware. The example crew already has memory off, and when you create your own crew in Part 3 you answer n when CrewAI asks Enable crew memory?.

> **NOTE**
>
> To keep separate caches, for example one per crew or one per test, give each its own offload_folder_name. Change that one line in the agent's .jsonc file and run the crew again.

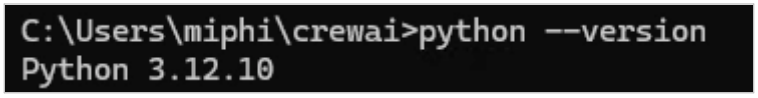
Part 3: Installing CrewAI and Creating a Crew

In this Part you install CrewAI 1.15.23 by hand with the uv package manager, create a new crew with the crewai create crew command, and connect its agent to aiDAPTIV+. Run the commands in PowerShell, opened the normal way (not with Run as administrator).

Check the Python version

CrewAI needs Python 3.10 or later, but earlier than 3.14 (Python >=3.10 and <3.14). Check the version you have:

```
python --version
```



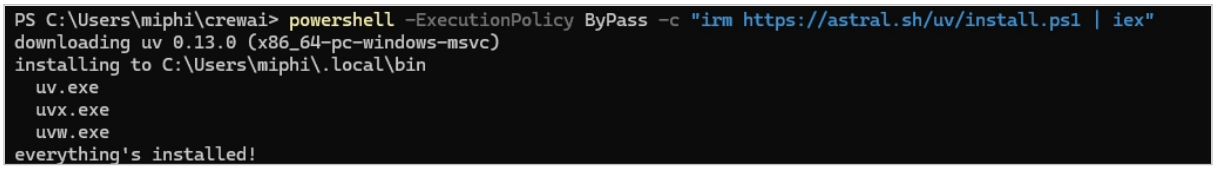
Python 3.12.10 is installed, which CrewAI supports

If Python is not found, or the version is outside this range, install Python 3.12 or 3.13 from python.org and open a new PowerShell window.

Install uv

uv is the Python package manager CrewAI uses. It installs CrewAI itself and manages each crew's virtual environment and package versions, much like npm does for Node.js. Install it with:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```



uv is downloaded and installed into your user folder

When it finishes, close and reopen PowerShell so the uv command is found.

Install CrewAI

Install CrewAI version 1.15.23 as a uv tool, so the crewai command is available in any terminal:

```
uv tool install crewai==1.15.23
```

```
PS C:\Users\miphi\crewai> uv tool install crewai==1.15.23
Resolved 140 packages in 1.59s
Prepared 140 packages in 10.13s
Installed 140 packages in 1.25s
 + aiofiles==24.1.0
 + aiohappyeyeballs==2.7.1
 + aiohttp==3.14.5
 + aiosignal==1.4.0
 + aiosqlite==0.21.0
 + annotated-doc==0.0.5
 + annotated-types==0.8.0
 + anyio==4.15.1
 + appdirs==1.4.4
```

uv resolves and installs CrewAI 1.15.23 and its packages

> **NOTE**
> Install this exact version. The example crew and the steps in this guide were tested with CrewAI 1.15.23.

Set up the PATH and check the version

If uv prints a PATH warning after installing CrewAI, add its tools folder to your PATH, then close and reopen PowerShell:

```
uv tool update-shell
```

```
PS C:\Users\miphi\crewai> uv tool update-shell
Executable directory C:\Users\miphi\.local\bin is already in PATH
```

In this example the uv tools folder is already on the PATH

Then confirm which version of CrewAI is installed:

```
uv tool list
```

```
PS C:\Users\miphi\crewai> uv tool list
crewai v1.15.23
- crewai
```

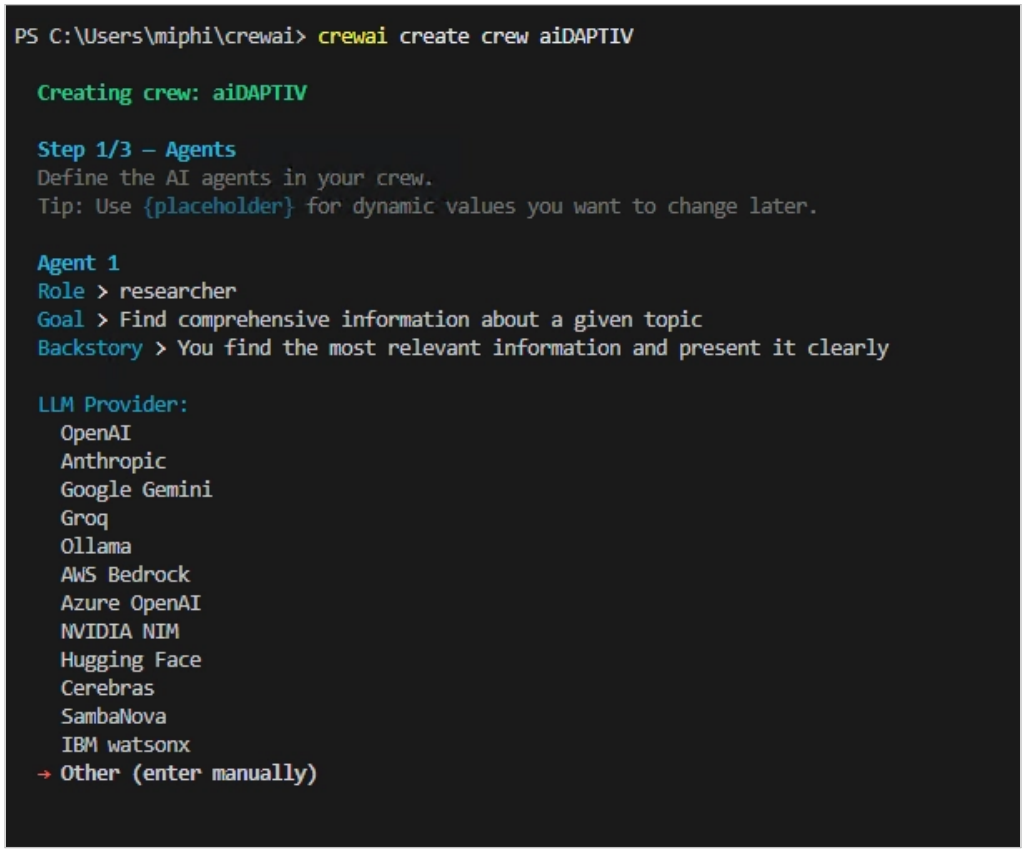
CrewAI v1.15.23 is installed and provides the crewai command

Create a crew

1. Go to the folder where you want the crew, then create it. In this example the crew is called aiDAPTIV:

```
crewai create crew aiDAPTIV
```

2. At Step 1/3, Agents, enter the agent's role, goal and backstory, then choose Other (enter manually) as the LLM provider.



Agent questions: role, goal and backstory, then the LLM provider list

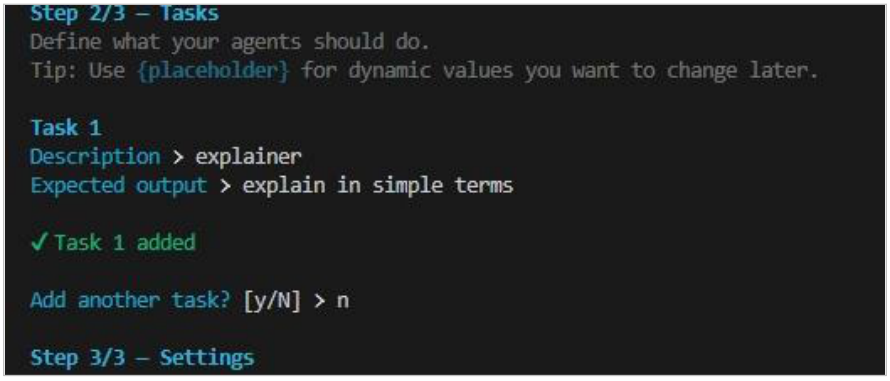
3. When asked for the model, type any name, for example aiDAPTIV. You replace this with the aiDAPTIV+ llm block later in this Part, so the value does not matter.



The model name typed for the Other provider

4. Answer the remaining questions. Press Enter to accept a default, or type your own value:

Question	We answered	What it means
Role	researcher	The agent's job title. It also names the agent's settings file, agents\researcher.jsonc.
Goal	Find comprehensive information about a given topic	What the agent is trying to achieve.
Backstory	You find the most relevant information and present it clearly	Background that shapes how the agent works and writes.
LLM Provider	Other (enter manually), then aiDAPTIV	A placeholder, replaced by the aiDAPTIV+ llm block in the next steps.
Tools	No tools	Press Enter to confirm without selecting any tools.
Enable step-by-step planning?	n	Planning adds extra calls to the model. It is not needed for this example.
Allow delegation to other agents?	n	Lets agents hand work to each other in a hierarchy. Answer y only if your crew needs it.
Add another agent?	n	Answer y to add more agents to the crew.
Task description	explainer	What the task asks the agent to do. Use {placeholder} for values you want to supply each time the crew runs.
Expected output	explain in simple terms	What a good result looks like.
Add another task?	n	Answer y to add more tasks.
Enable crew memory?	n	Keeps every call on your own hardware. See Why crew memory is turned off in Part 2.



Step 2/3, Tasks: the task description and expected output

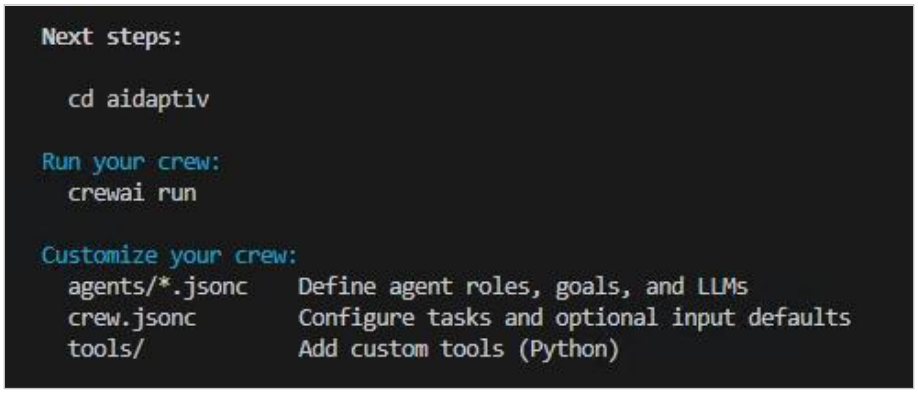


Step 3/3, Settings: crew memory turned off

5. CrewAI confirms that the crew was created and shows the next steps:



The crew has been created

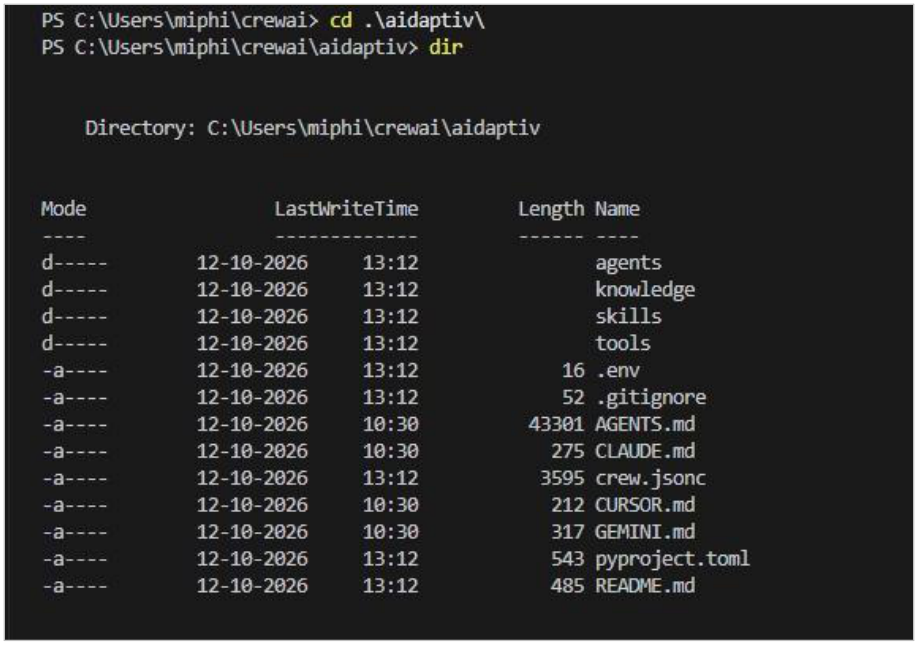


The next steps shown by CrewAI

Look inside the crew folder

CrewAI writes the folder name in lower case, so aiDAPTIV becomes aidaptiv. Go into the folder and list its contents:

```
cd aidaptiv
dir
```



The files and folders of the new aidaptiv crew

The crew folder is laid out like this:

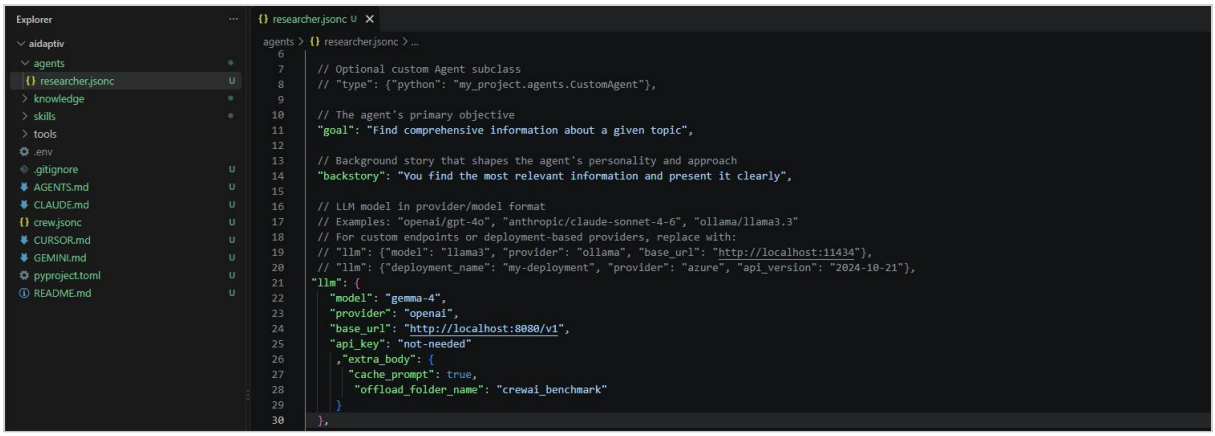
```
aidaptiv/
├── .gitignore
├── .env
├── agents/
│   └── researcher.jsonc
├── crew.jsonc
├── knowledge/
├── pyproject.toml
├── README.md
├── skills/
└── tools/
```

File	What it does
crew.jsonc	Configures the crew without any boilerplate Python code: its tasks, the order they run in, its inputs and settings such as "memory". It is a JSON with Comments file.
agents\researcher.jsonc	Defines a single agent: its role, goal, backstory and the llm block that says which model it uses. A crew has one file per agent.

Connect the agent to aiDAPTIV+

Open agents\researcher.jsonc in a text editor and replace its llm entry with the aiDAPTIV+ block from Part 2:

```
"llm": {
  "model": "gemma-4",
  "provider": "openai",
  "base_url": "http://localhost:8080/v1",
  "api_key": "not-needed",
  "extra_body": {
    "cache_prompt": true,
    "offload_folder_name": "crewai_benchmark"
  }
},
```



The aiDAPTIV+ llm block in aidaptiv\agents\researcher.jsonc

Set model to the name your server reports at /v1/models if you want to be safe (Part 1), and offload_folder_name to the cache folder you want for this crew. Save the file. Your crew is now configured for aiDAPTIV+.

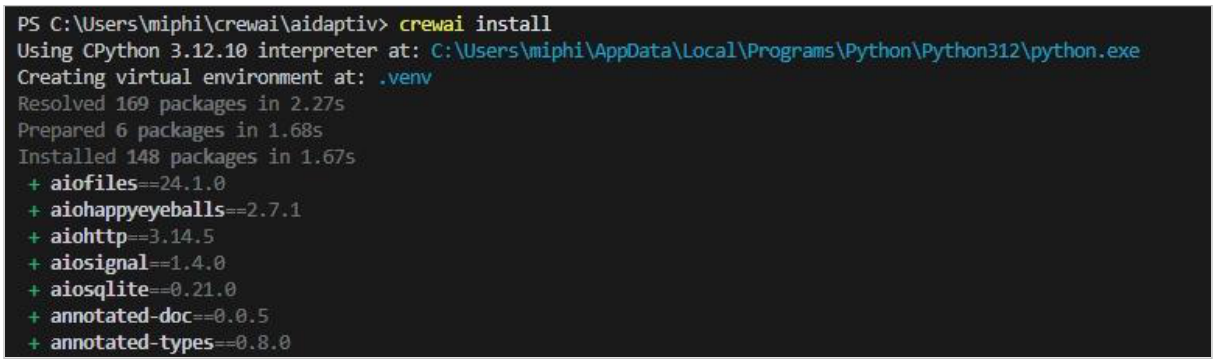
> **NOTE**
>
> Add this block to every agent file in the crew. An agent without it still points at the placeholder provider entered during crewai create crew and will not reach your aiDAPTIV+ server.

Part 4: Running a Crew

Run your new crew

1. In the crew folder, create the virtual environment (.venv) and install the crew's dependencies. The first time, this can take a minute or two:

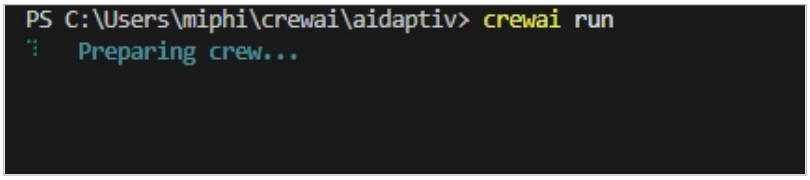
```
crewai install
```



crewai install creates the virtual environment and installs the dependencies

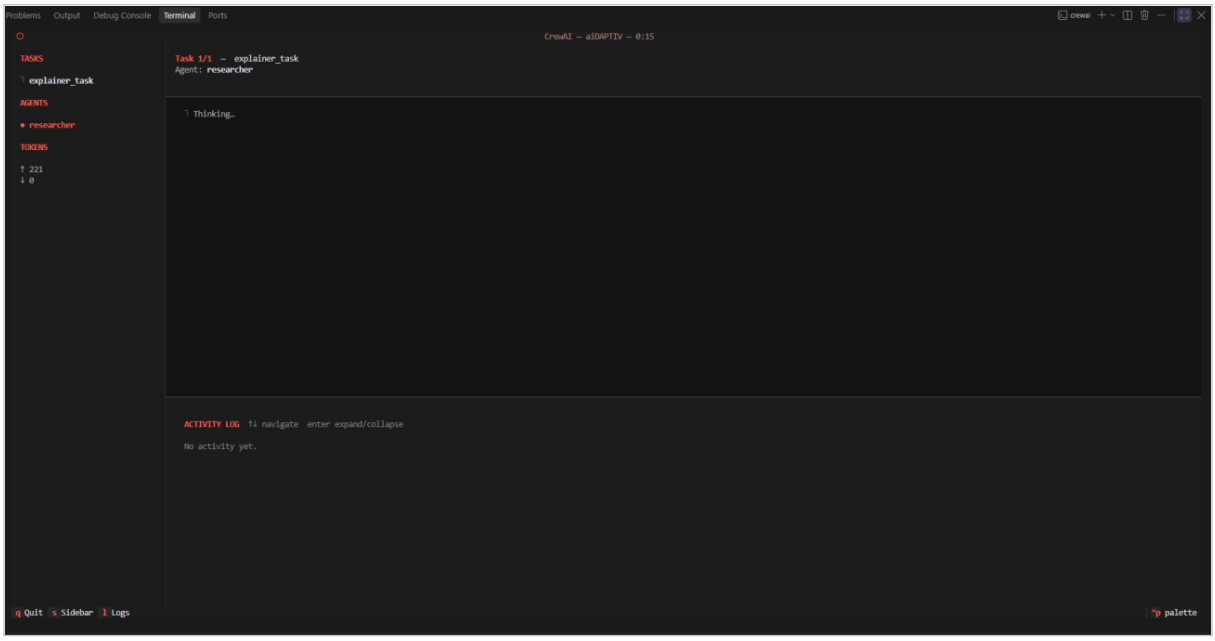
2. Start the crew:

```
crewai run
```



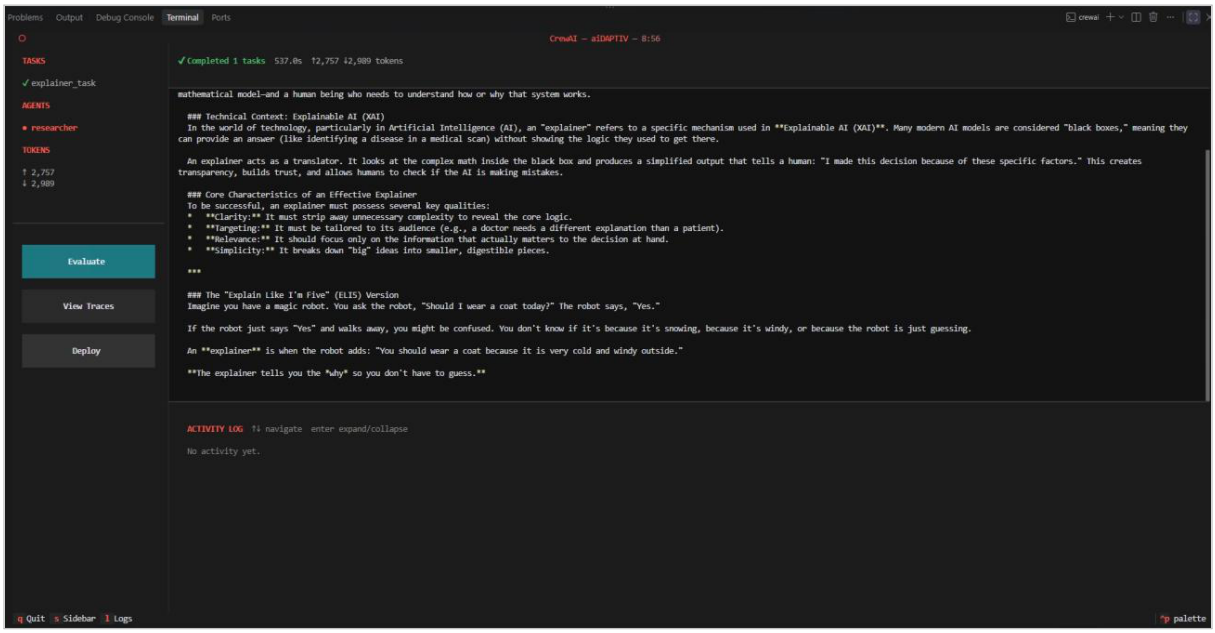
crewai run preparing the new crew

3. CrewAI opens a live screen that shows the task, the agent and the tokens used while the model works.



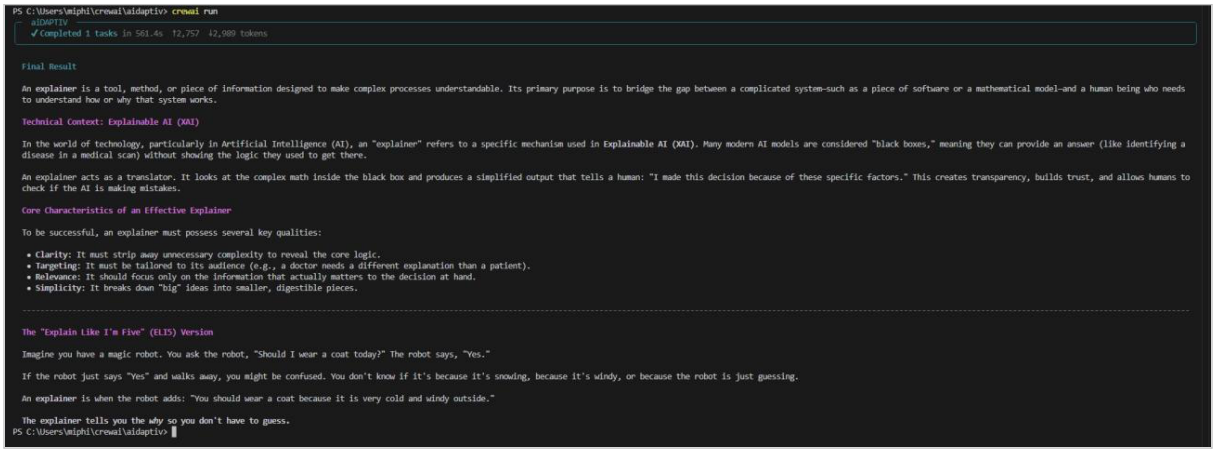
The crew at work, waiting for the model's reply

4. When the task finishes, the result appears in the middle of the screen.



Task completed, with the result and the token counts

5. Press q to quit. The final result is also printed in the terminal.



The final result in the terminal

You can now change the agents, tasks and inputs to suit your own work. A new crew has no document or question to work on yet, so in this example the agent simply explained the word in its task description. The example crew below shows how to give a crew a document and a question.

Run the example crew with a test document

The example crew has one agent, developer, and one task, explainer. The task is built from two inputs: a document and a question. The run.py helper fills in both, so you can try the crew on different documents without editing any files.

1. Go to the example folder inside the repository:

```
cd aiDAPTIV-Integration-crewAI\aiDAPTIV_Example_Project
```

```
C:\Users\miphi\crewAI\aiDAPTIV_Example_Project>dir
 Volume in drive C has no label.
 Volume Serial Number is D679-B79B

 Directory of C:\Users\miphi\crewAI\aiDAPTIV_Example_Project

14-10-2026  16:23    <DIR>          .
14-10-2026  16:23    <DIR>          ..
14-10-2026  16:23                52 .gitignore
14-10-2026  16:23    <DIR>          agents
14-10-2026  16:23            44,516 AGENTS.md
14-10-2026  16:23               282 CLAUDE.md
14-10-2026  16:23            79,285 crew.jsonc
14-10-2026  16:23         3,588,221 crew.log.txt
14-10-2026  16:23               219 CURSOR.md
14-10-2026  16:23                23 env.example
14-10-2026  16:23               325 GEMINI.md
14-10-2026  16:23    <DIR>          knowledge
14-10-2026  16:23            60,455 prompt_10k.txt
14-10-2026  16:23            78,484 prompt_13k.txt
14-10-2026  16:23            48,421 prompt_8k.txt
14-10-2026  16:23               541 pyproject.toml
14-10-2026  16:23               484 README.md
14-10-2026  16:23               513 run.py
14-10-2026  16:23    <DIR>          skills
14-10-2026  16:23           802,751 uv.lock
              15 File(s)      4,704,572 bytes
               5 Dir(s)  486,595,653,632 bytes free
```

The example crew, with the run.py helper and the three test documents

2. Choose a document and a question with run.py. Put the question in double quotes:

```
python run.py prompt_10k.txt "in simple terms"
```

3. run.py reads the document, writes it and your question into the inputs section of crew.jsonc and confirms with inputs updated. Then start the crew:

```
C:\Users\miphi\crewAI\aiDAPTIV_Example_Project>python run.py prompt_10k.txt "in simple terms "
inputs updated: prompt_10k.txt | in simple terms

Now you can Run the CrewAI run !

C:\Users\miphi\crewAI\aiDAPTIV_Example_Project>crewai run
```

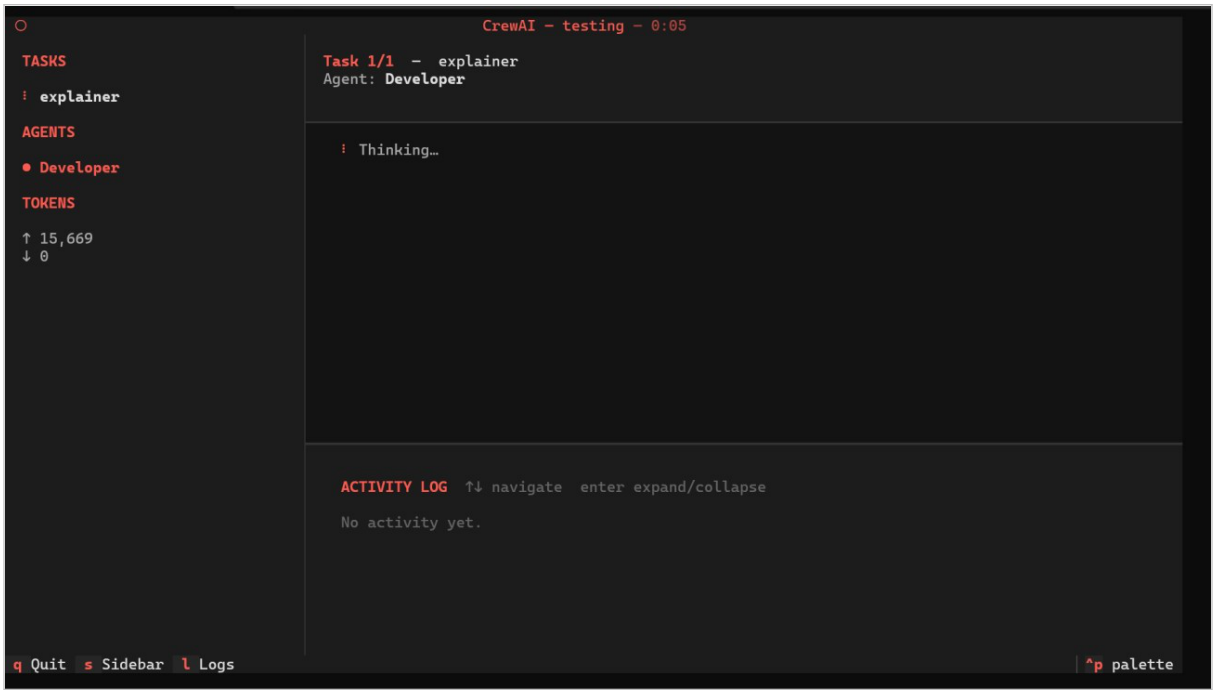
run.py sets the inputs, then crewai run starts the crew

4. As with your own crew, the first run installs the dependencies before the crew starts.

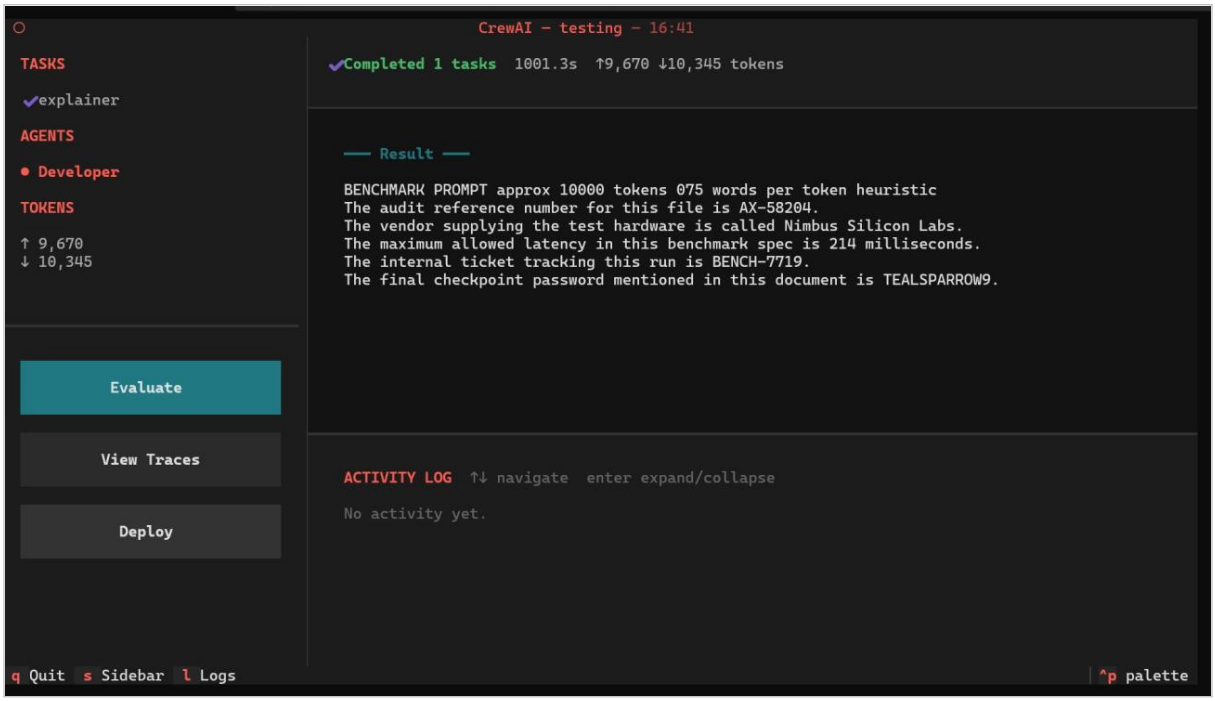
```
C:\Users\miphi\crewAI\aiDAPTIV_Example_Project>crewai run
Syncing dependencies from lockfile...
Using CPython 3.13.16
Creating virtual environment at: .venv
Prepared 3 packages in 2.21s
[75/147] uvicorn==0.54.0
```

crewai run preparing the example crew

5. The developer agent works on the explainer task and the result appears when it finishes.

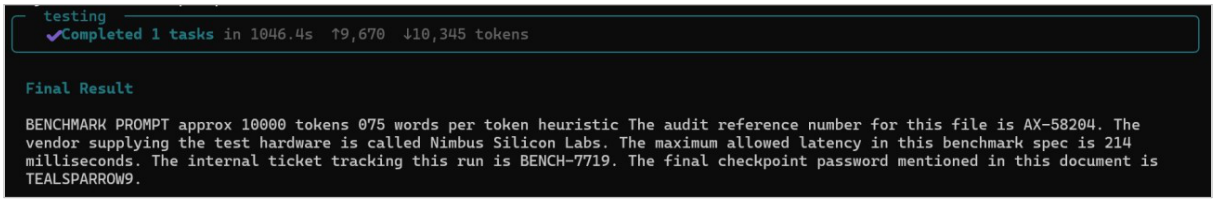


The developer agent working on the explainer task



Task completed, with the total time and the token counts

6. Press q to quit and read the final result in the terminal.



The final result of the example crew

The three test documents:

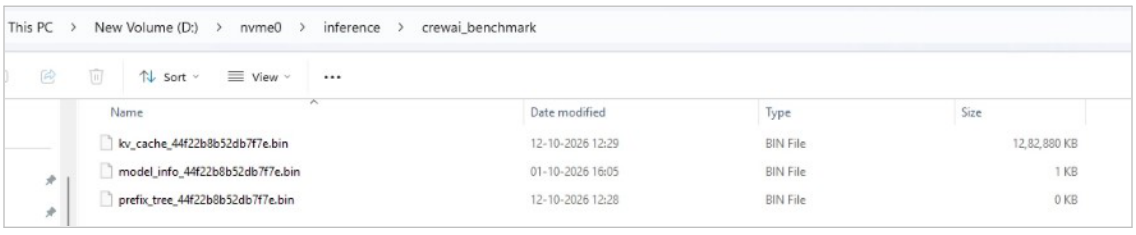
Document	Prompt size sent to the model
prompt_8k.txt	About 7,800 tokens
prompt_10k.txt	About 9,700 tokens
prompt_13k.txt	About 12,400 tokens

> **NOTE**
> To ask another question about the same document, run run.py again with the new question and then crewai run. The document comes first in the prompt and has not changed, so aiDAPTIV+ can reuse its saved notes and the answer starts much sooner. Part 5 shows how to confirm this.

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After running a crew at least once, browse to your aiDAPTIV NVMe cache root plus the offload_folder_name from the agent's llm block (crewai_benchmark for the example crew). On our test PC this is D:\nvme0\inference\crewai_benchmark. New files there confirm that the KV cache is being written to the aiDAPTIV SSD, not just held in graphics memory.



KV-cache files in the crewai_benchmark folder on the aiDAPTIV SSD

Confirm a cache hit

Run the crew once so that its KV cache is saved. Then run it again on the same document, even after restarting the model server, and check the llama-server log for the new request:

```
0.59.047.970 I srv  params_from_: Chat format: peg-gemma4
0.59.051.262 I slot get_availabl: id  3 | task -1 | selected slot by LRU, t_last = -1
0.59.051.437 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.59.051.641 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.59.640.147 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.59.640.163 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 12440
[MDW][Info][Runtime] Save 0 tokens to folder 'crewai_benchmark'
```

Hit token cnt (CACHE): 12440 on the repeat run

A large, non-zero Hit token cnt (CACHE) value is direct proof that those tokens were loaded from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. A closing line such as "Save 0 tokens to folder 'crewai_benchmark'" confirms that nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster time to first token on repeated or overlapping prompts.

If your second run still shows Hit token cnt (CACHE): 0

Check that cache_prompt and offload_folder_name are inside extra_body in the agent's llm block, that the second run uses the same document (the question can change), and that the cache folder has free space on the aiDAPTIV SSD.

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit

Run 2: Cache Hit (KV Reuse)

From our benchmarking test with prompt_13k.txt. The first run had already saved the KV cache for this prompt, the model server was then restarted, and this run reused the cache:

```
[MDW][Info][Runtime] Save 312 tokens to folder 'crewai_benchmark'
1.32.175.696 I slot print_timing: id  3 | task 0 | prompt eval time =    1233.66 ms /     5 tokens (  246.73 ms per token,     4.05 tokens per second)
1.32.175.700 I slot print_timing: id  3 | task 0 |       eval time =   31432.53 ms /   317 tokens (   99.16 ms per token,    10.09 tokens per second)
1.32.175.702 I slot print_timing: id  3 | task 0 |      total time =   32666.20 ms /   322 tokens
1.32.175.704 I slot print_timing: id  3 | task 0 |    graphs reused =        314
1.32.177.030 I slot      release: id  3 | task 0 | stop processing: n_tokens = 12761, truncated = 0
1.32.177.064 I srv  update_slots: all slots are idle
```

Timing lines from the llama-server log for the repeat run

TTFT: 1,233.66 ms • TPS: 10.09 tokens/sec • Hit Cache (CACHE): 12,440 tokens

Metric	Run 2, warm (cache hit)
Hit token cnt (CACHE)	12,440 tokens
TTFT (prompt eval time)	1,233.66 ms
TPS (decode / eval speed)	10.09 tokens/sec

Only 5 new tokens had to be processed, so the prompt eval time is the time to the first token. The other 12,440 tokens came from the aiDAPTIV+ cache. The eval line gives the speed at which the answer was written. Without aiDAPTIV+, the same prompt took 4 minutes 38 seconds before the first token.

Benchmark your own system

1. Set offload_folder_name in the agent's llm block to a new folder name, so the first run starts without a saved cache.
2. Run the example crew with prompt_13k.txt and note the prompt eval time in the server log. This is the first read.
3. Restart the model server, run the crew again with the same document and note the prompt eval time, the eval speed and the Hit token cnt (CACHE) value.
4. For the without-aiDAPTIV+ figure, run the same document against a standard llama.cpp server with the same model.

Part 7: Troubleshooting

These are the messages you are most likely to see, and what to do about each one.

Message or symptom	What to do
python is not recognized, or the version is not 3.10 to 3.13	Install Python 3.12 or 3.13 from python.org, tick Add python.exe to PATH during setup, and open a new PowerShell window.
uv is not recognized	Close and reopen PowerShell so it picks up the updated PATH. If it is still not found, run the uv install command from Part 3 again.
crewai is not recognized, or uv shows a PATH warning	Run uv tool update-shell, then close and reopen PowerShell.
uv tool list shows a CrewAI version other than 1.15.23	Run uv tool install crewai==1.15.23 --force to replace it with the tested version.
The crew asks for an API key or tries to reach another provider	The agent's .jsonc file still has the placeholder provider from crewai create crew. Replace its llm entry with the aiDAPTIV+ block, as in Part 3.
crewai run fails with "Model not found"	Change "model" in the agent's .jsonc file to the exact name the server reports at /v1/models.
crewai run fails with a connection error	The model server is not running, or base_url in the agent's llm block points at the wrong address or port. Start the aiDAPTIV+ middleware and the llama.cpp server, and check the base_url.
The agent asks for a topic instead of answering	The crew has no inputs yet. Use run.py as in Part 4, or add a {placeholder} to the task description and supply its value in the inputs section of crew.jsonc.
Hit token cnt (CACHE): 0 on a repeat run	See the note at the end of Part 5.